

Structures

Objectives

After studying this section, you should be able to:

- ◆ What is C structure?
- ◆ When to use structures.
- ◆ Syntax of a structure.
- ◆ How to declare variable of type structure?
- ◆ Fields of a structure and how to initialize them.
- ◆ How to manipulate structure type

Contents

1. Structure Definition
2. Struct Syntax
3. Manipulating Structure Types
4. Arrays of Structures
5. Function with a Structure Input Parameter

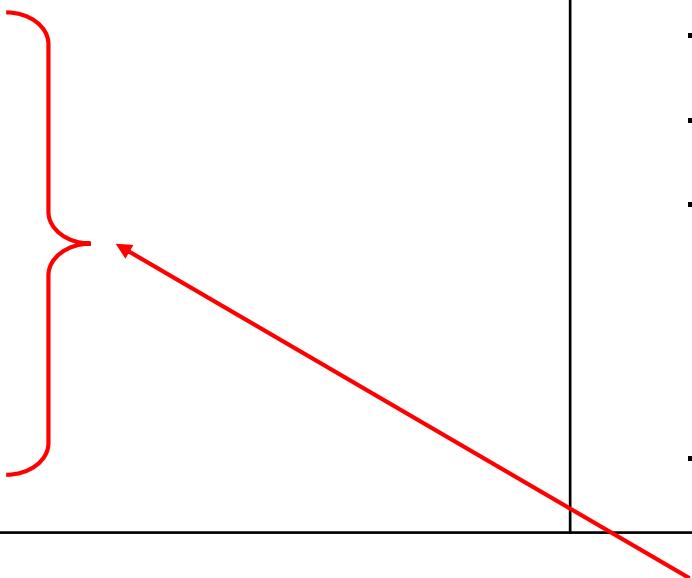
1. Structure Definition

- ◆ The structure in C is a user-defined data type that can be used to group items of possibly different types into a single type.
- ◆ Structures are also called records.
- ◆ The **struct** keyword is used to define the structure in the C programming language.
- ◆ Unlike arrays, a struct is composed of data of different types.
- ◆ You use structures to group data that belong together.
- ◆ Additionally, the values of a structure are stored in contiguous memory locations.

Structure Definition (cont.)

◆ Examples:

Student information:	Bank account information:
- student id, - last name, - first name - major, - gender, ...	- account number, - account type - account holder + first name + last name - balance



- ◆ Data elements in a structure are called **fields** or **members**.
- ◆ Complex data structures can be formed by defining arrays of structs.

2. Struct Syntax

- ◆ Syntax of the structure type:

```
typedef struct{  
    dataType1 field1;  
    dataType2 field2;  
    ...  
} structName;
```

Or

```
struct structName{  
    dataType1 field1;  
    dataType2 field2;  
    ...  
};
```

Syntax Structure (cont.)

- ◆ Examples:

Example 1	Example 2	Example 3
typedef struct { int day; int month; int year; } eventDate ;	typedef struct { char name[20]; int age; } person ;	struct telephone { char name[30]; int number; };

- ◆ How to declare variable of type structure?

```
structName variableName;
```

- ◆ Example: eventDate ev; person p; telephone tel;

3. Manipulating Structure Types

◆ How to access a field in a structure:

- Use the direct component selection operator, which is a **period** ‘.’
- The ‘.’ operator has the highest priority in the operator precedence.
- Example:

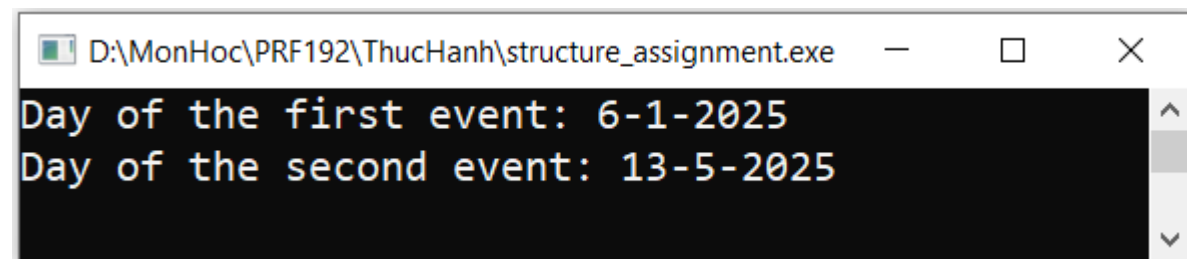
```
person p1;  
p1.name;  
p1.age;
```

◆ Structure assignment:

- The copy of an entire structure can be easily done by the assignment operator.
- Each component in one structure is copied into the corresponding component in the other structure.

Example 1: Structure Type

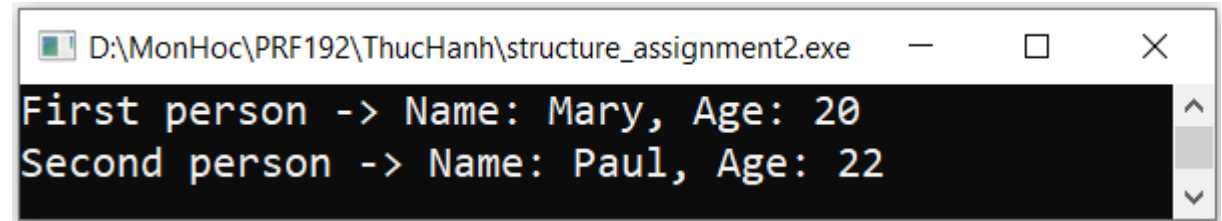
```
1  #include <stdio.h>
2
3  typedef struct{
4      int day;
5      int month;
6      int year;
7  } eventDate;
8
9  int main(){
10     // Declare a variable with struct type
11     eventDate ev1;
12
13     // Assignment value into the members of 'ev1'
14     ev1.day = 6;
15     ev1.month = 1;
16     ev1.year = 2025;
17
18     // Or: Declare and Initialization
19     eventDate ev2 = {13, 05, 2025};
20
21     printf("Day of the first event: %d-%d-%d\n", ev1.day, ev1.month, ev1.year);
22     printf("Day of the second event: %d-%d-%d\n", ev2.day, ev2.month, ev2.year);
23
24     return 0;
25 }
```



```
D:\MonHoc\PRF192\ThucHanh\structure_assignment.exe
Day of the first event: 6-1-2025
Day of the second event: 13-5-2025
```

Example 2: Structure Type

```
1  #include <stdio.h>
2  #include <string.h>
3
4  struct person{
5      char name[20];
6      int age;
7  };
8
9  int main(){
10     // Declare and Initialization
11     struct person p1 = {"Mary", 20};
12     struct person p2;
13     strcpy(p2.name, "Paul"); // Copy 'Paul' to name of p2
14     p2.age = 22;
15
16     printf("First person -> Name: %s, Age: %d\n", p1.name, p1.age);
17     printf("Second person -> Name: %s, Age: %d\n", p2.name, p2.age);
18
19     return 0;
20 }
```



D:\MonHoc\PRF192\ThucHanh\structure_assignment2.exe

```
First person -> Name: Mary, Age: 20
Second person -> Name: Paul, Age: 22
```

4. Arrays of Structures

- ◆ We can also declare an array of structures.
- ◆ Recall the syntax of an array: `dataType array_name[size];`
 - **dataType** can any C type including struct type
- ◆ The array of structures can be simply manipulated as arrays of simple data types.

- ◆ **Example:**

Write a program to organize information, includes: **name** and **age** of three family members. Print out the information for all members on each line.

Example 1: Array of Structures

```
1 #include <stdio.h>
2 #include <string.h>
3 typedef struct{
4     char name[20];
5     int age;
6 } person;
7
8 int main(){
9     // Declara an array of structure
10    person family[3];
11
12    strcpy(family[0].name, "John"); family[0].age = 30;
13    strcpy(family[1].name, "Sara"); family[1].age = 28;
14    strcpy(family[2].name, "David"); family[2].age = 3;
15
16    printf("Father -> Name: %s, Age: %d\n", family[0].name, family[0].age);
17    printf("Mother -> Name: %s, Age: %d\n", family[1].name, family[1].age);
18    printf("Son -> Name: %s, Age: %d\n", family[2].name, family[2].age);
19
20    return 0;
21 }
```

D:\MonHoc\PRF192\ThucHanh\arrayOfStructure.exe

```
Father -> Name: John, Age: 30
Mother -> Name: Sara, Age: 28
Son -> Name: David, Age: 3
```

family	name	age
family[0]	John	30
family[1]	Sara	28
family[2]	David	2

family[2].name

family[1].age

5. Function with a Structure Input Parameter

- ◆ When a structure variable is passed as an input argument to a function, all its component values are copied into the local structure variable.
- ◆ **Syntax:**

```
dataType functionName(structName parameter){ ... }
```

- ◆ **Example:**

Write a C program to print a student's information including: id, name, age. Use a structure to define the student's data structure and use a function with a parameter of type structure to print the information.

Function with a Structure Input Parameter: Example 2

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  typedef struct{
6      int id;
7      char name[20];
8      char grade;
9  } student;
10
11 void printStudent(student s){
12     printf("Id: %d, Name: %s, Grade: %c\n", s.id, s.name, s.grade);
13 }
14
15 int main(){
16     // Declaration a variable with structure type
17     student s1;
18     // Assign the value into members of structure
19     s1.id = 1001; strcpy(s1.name, "Tom"); s1.grade = 'A';
20     // Print student
21     printf("Student Information:\n");
22     printStudent(s1);
23
24     system("pause");
25     return 0;
26 }
```

```
Student Information:
Id: 1001, Name: Tom, Grade: A
Press any key to continue . . .
```

Case Study:

- ◆ Write a C program that allows users to manage account information of up to 100 customers. Customer account information includes: **accountNumber** (int); **accountType** (char[20]); **accountHolderName** (char[40]); **balance** (double).
- ◆ **Requirements:**
 - Enter customer account information from the keyboard
 - Print out a list of customer account information
 - Find and print out information on customers with a balance greater than \$1000

Case Study: Code design

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  typedef struct{
6      int accountNumber;
7      char accountType[20];
8      char accountHolderName[40];
9      double balance;
10 } accountCustomer;
11
12 void clear(void){
13     while(getchar()!='\n');
14 }
```

```
16 // Prototypes:
17 void inputInfo(accountCustomer accounts[], int n);
18 void printAccount(accountCustomer acc);
19 void printInfo(accountCustomer accounts[], int n);
20 void searchAccountCustomers(accountCustomer accounts[], int n);
21
22 int main(){
23     accountCustomer accountCustomers[100];
24     int n;
25
26     // Input n
27     printf("Input number of the Customer account: ");
28     scanf("%d", &n);
29     inputInfo(accountCustomers, n);
30     printInfo(accountCustomers, n);
31     searchAccountCustomers(accountCustomers, n);
32     system("pause");
33     return 0;
34 }
```


Case Study: Code design

```
36 void inputInfo(accountCustomer accounts[], int n){
37     int i;
38     for(i=0; i<n; i++){
39         printf("# %d\n", i+1);
40         printf("Account number: "); scanf("%d", &accounts[i].accountNumber);
41         clear();
42         printf("Account type: "); scanf("%[^\\n]", &accounts[i].accountType);
43         clear();
44         printf("Account holder name: "); scanf("%[^\\n]", &accounts[i].accountHolderName);
45         printf("Balance: "); scanf("%lf", &accounts[i].balance);
46         clear();
47     }
48 }
49
50 void printAccount(accountCustomer acc){
51     printf("%d\\t%s\\t%s\\t%.2lf\\n", acc.accountNumber, acc.accountType, acc.accountHolderName, acc.balance);
52 }
```

Case Study: Code design

```
54 void printInfo(accountCustomer accounts[], int n){  
55     int i;  
56     printf("\nList of the Customer account:\n");  
57     for(i=0; i<n; i++){  
58         printAccount(accounts[i]);  
59     }  
60 }  
61  
62 void searchAccountCustomers(accountCustomer accounts[], int n){  
63     int i;  
64     printf("\nSearch result:\n");  
65     for(i=0; i<n; i++){  
66         if(accounts[i].balance>1000){  
67             printAccount(accounts[i]);  
68         }  
69     }  
70 }
```

Case Study: Compile & Run

```
Input number of the Customer account: 3
# 1
Account number: 1001
Account type: Checking
Account holder name: Pham Ngoc Tho
Balance: 10050.123
# 2
Account number: 1002
Account type: Saving
Account holder name: Hoang Duc Binh
Balance: 500
# 3
Account number: 1003
Account type: Checking
Account holder name: Pham Minh Chau
Balance: 6666.888

List of the Customer account:
1001    Checking    Pham Ngoc Tho    10050.12
1002    Saving    Hoang Duc Binh    500.00
1003    Checking    Pham Minh Chau    6666.89

Search result:
1001    Checking    Pham Ngoc Tho    10050.12
1003    Checking    Pham Minh Chau    6666.89
Press any key to continue . . .
```

Summary

- ◆ Understanding what a Structure type is?
- ◆ When to use a Structure type in a program
- ◆ How to define a Structure type and use a Structure in a program
- ◆ Know how to work with the components of a Structure